

Data Modeling in OLAP Systems

Introduction to OLAP Systems

Alright, let's start by getting a understanding of what OLAP systems are and why they're so important. Imagine you're running a business, and you have tons of data coming in every day—sales transactions, customer interactions, inventory changes, and so on. Now, if you want to make smart decisions, like figuring out what products to stock up on or which customers to target, you need to analyze all this data.

That's where **OLAP**—which stands for **Online Analytical Processing**—comes into play. These systems are designed to help you dive deep into your data, letting you run complex queries, generate reports, and basically pull out the insights you need to make informed decisions. OLAP systems are like the brain behind your business intelligence, turning raw data into actionable insights.

OLAP vs. OLTP: What's the Difference?

Now, you might be wondering how OLAP systems are different from something called **OLTP**—**Online Transaction Processing** systems. Think of OLTP as the workhorse that handles the day-to-day operations of your business. It's like the cash register in your store, processing each transaction quickly and efficiently.

But OLAP is different. It's not about handling individual transactions in real time; instead, it's about looking at the bigger picture. OLAP systems take all those transactions, store them, and then let you analyze them to spot trends, patterns, and opportunities. So, while OLTP is all about speed and accuracy in processing transactions, OLAP is about deep, insightful analysis over time.

Diving Into OLAP Data Modeling

Now that we've got the basics down, let's talk about how we actually structure the data in an OLAP system. This is where data modeling comes into play, and it all starts with two key players: **Fact Tables** and **Dimension Tables**.

1. Fact Tables: The Heart of Your Data Model

Imagine you're managing a chain of retail stores, and you want to analyze your sales data. The **Fact Table** is where the action happens—it's the place where all your core data is stored. Think of it as the table that holds all the juicy details about your business metrics, like sales, revenue, or quantities sold.

Hence, a fact table is a table that primarily stores quantitative data—often referred to as measures—that can be analyzed. These measures typically represent business metrics like sales, revenue, or quantities, which are the focus of queries and analysis. The fact table contains the core data you want to analyze and the meaning of this data is represented by a dimension table

Here's an example to make it clearer: Let's say you have a table called **SalesFact** in your data warehouse. It might look something like this:

Date	ProductID	StoreID	SalesAmount	QuantitySold
2024-08-01	1001	1	500	10
2024-08-01	1002	2	300	5
2024-08-02	1003	1	450	8

So, what are we looking at here? **SalesAmount** and **QuantitySold** are what we call **measures**—they're the numbers you want to analyze. The other columns, like **ProductID** and **StoreID**, are **foreign keys** that connect this fact table to other tables (which we'll talk about in a bit).

Key Takeaways About Fact Tables:

- **Foreign Keys:** These are the links that connect your fact table to other tables, giving your data context.
- **Measurable Data:** Fact tables store the numbers—like sales, revenue, or quantities—that you want to analyze.

- **Lots of Rows:** Fact tables usually have a ton of rows because they capture every transaction or event in detail.
- **Grain:** The grain of a fact table is the level of detail it captures. In our example, each row represents a single sale.

2. Dimension Tables: Adding Context to Your Data

Alright, now let's talk about **Dimension Tables**. If the fact table is the heart of your data model, the dimension tables are like the veins and arteries—they provide the context that helps you make sense of the raw data in your fact table. They contain descriptive attributes (often referred to as dimensions) that relate to the fact table and help categorize, filter, and aggregate the data in meaningful ways.

Example Time: Let's go back to our **SalesFact** table. To really understand the sales data, you'd want to know more about the products, stores, and dates involved. That's where dimension tables come in.

You might have a **ProductDimension** table that looks like this:

ProductID	ProductName	Category	Brand
1001	Laptop	Electronics	TechCorp
1002	Smartphone	Electronics	PhoneCo
1003	Desk Chair	Furniture	ComfortPlus

And a **StoreDimension** table like this:

StoreID	StoreName	Location	Manager
1	Downtown	New York	John Smith
2	Mall Branch	Los Angeles	Sarah Brown

And a **DateDimension** table like this:

Date	DayOfWeek	Month	Quarter	Year
2024-08-01	Thursday	August	Q3	2024
2024-08-02	Friday	August	Q3	2024

These dimension tables help you drill down into the specifics. For example, you can now see that the **ProductID 1001** in the fact table is actually a **Laptop** from **TechCorp**.

Why Dimension Tables Matter:

- **Descriptive Data:** They store information like product names, categories, locations, and dates.
- **Primary Keys:** Each record in a dimension table has a unique identifier, which is referenced in the fact table.
- **Smaller Size:** Dimension tables usually have fewer rows than fact tables because they're summarizing data.
- **Enabling Analysis:** They allow you to filter, group, and label data in the fact table for more meaningful analysis.

Putting It All Together: Fact and Dimension Tables in Action

So how do fact and dimension tables work together? Let's say you want to find out, "What was the total sales amount for each product category in July 2024?" To answer that, you'd join the **SalesFact** table with the **ProductDimension** and **DateDimension** tables. You'd use the **DateDimension** to filter the records to July 2024, and the **ProductDimension** to group the results by product category.

Or maybe you're curious about, "Which store had the highest sales on August 1, 2024?" You'd join the **SalesFact** table with the **StoreDimension** and **DateDimension** tables to get your answer.

In both cases, the fact table gives you the numbers, and the dimension tables provide the context, letting you drill down into the data and get the insights you need.

Schema Design in OLAP: Star, Snowflake, and Galaxy

When you're building an OLAP system, how you organize your data is crucial. This is where schema design comes in. Let's explore a few common types of schemas: **Star Schema**, **Snowflake Schema**, and **Galaxy Schema**.

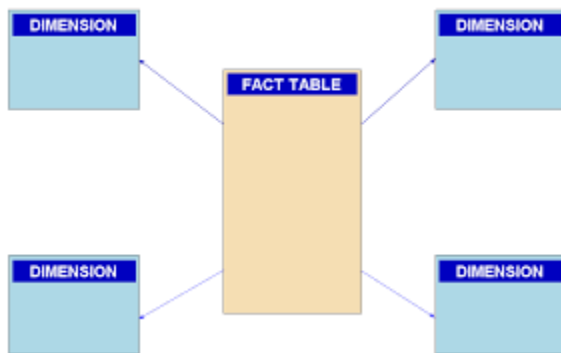
Star Schema: The Simple and Intuitive Choice

Imagine a star shape. In the middle, you have your fact table, and around it, like the points of a star, are your dimension tables. This is the **Star Schema**, and it's one of the simplest and most intuitive ways to structure your data.

Here's what it looks like:

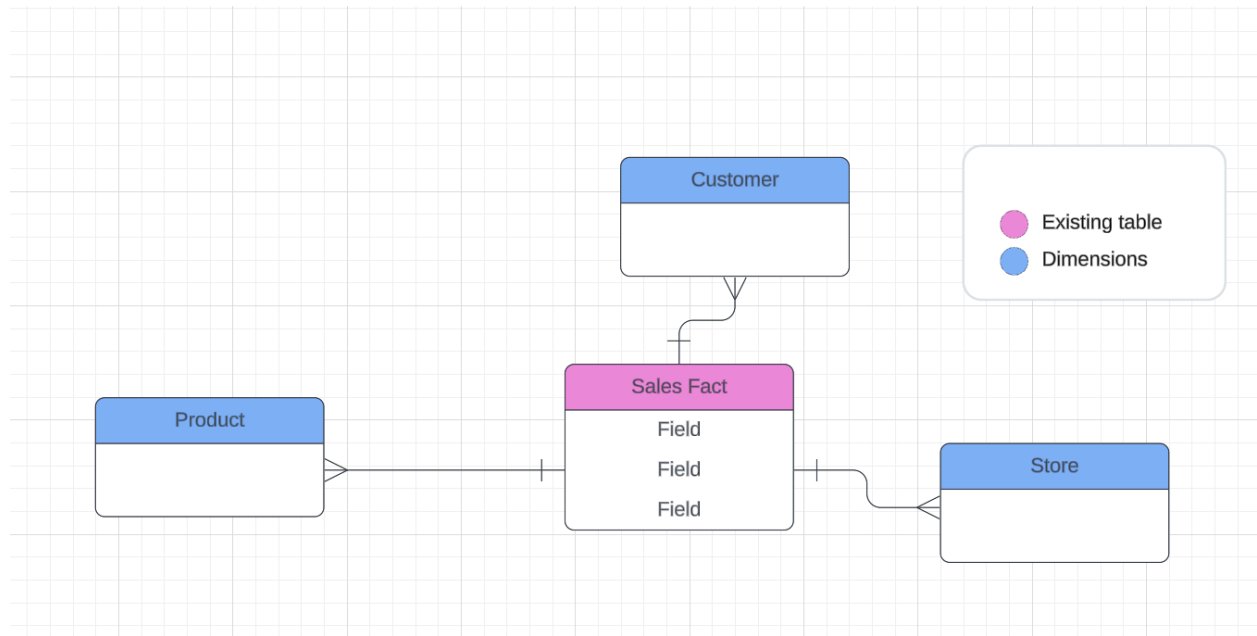
- **Fact Table:** This is the central table where all your measurable data lives—things like sales amounts, quantities sold, etc.
- **Dimension Tables:** These surround the fact table, providing context with details like product names, customer information, store locations

Visually, the fact table is in the center, and each dimension table is connected directly to it, like the points of a star.



Example:

Imagine you have a data warehouse for a retail store. The fact table **SalesFact** records sales data, while the dimension tables **ProductDimension**, **CustomerDimension**, and **StoreDimension** provide more details about the products, customers, and stores.



- **SalesFact:** Stores sales data (e.g., SalesAmount, QuantitySold).
- **ProductDimension:** Contains product details (e.g., ProductID, ProductName).
- **CustomerDimension:** Contains customer details (e.g., CustomerID, CustomerName).
- **StoreDimension:** Contains store details (e.g., StoreID, StoreLocation).

Why Use a Star Schema?

- **Easy to Understand:** Its straightforward structure makes it easy to design and query.
- **Fast Performance:** Queries are faster because there are fewer joins (connections between tables).
- **Some Redundancy:** Data might be duplicated in the dimension tables, but this trade-off helps with performance.

Snowflake Schema: When You Need More Normalization

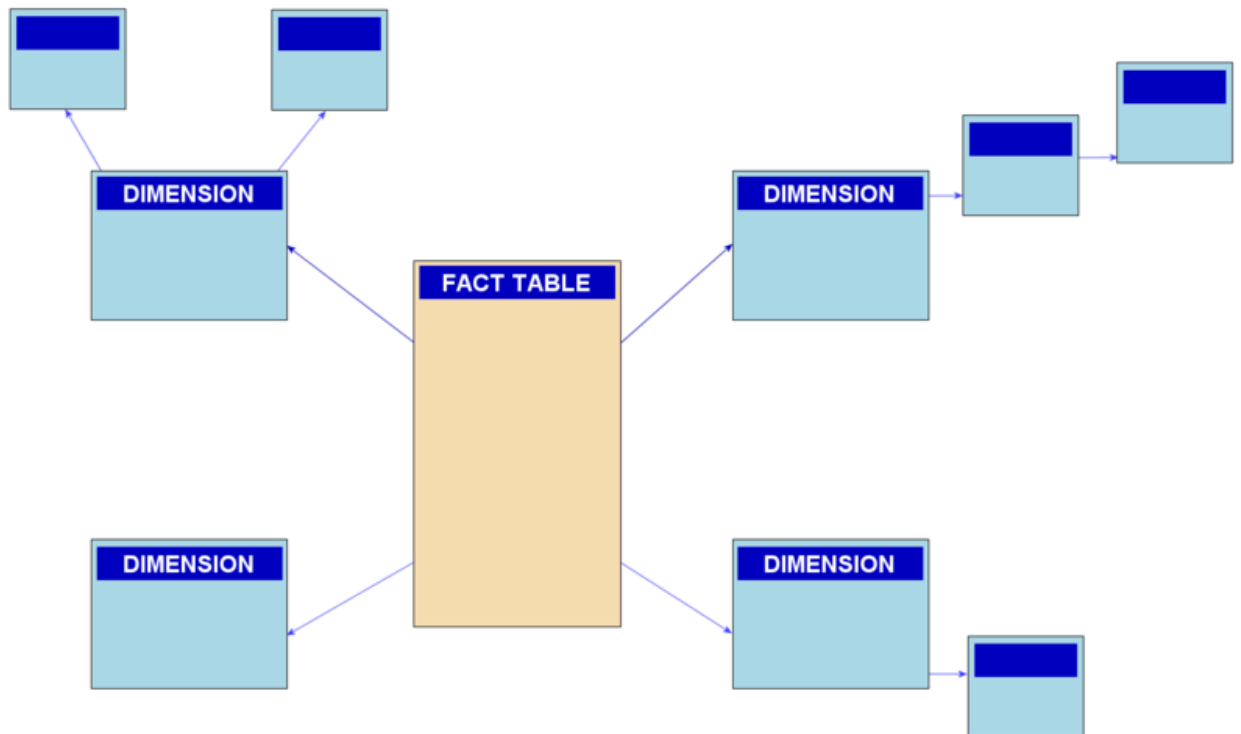
Now, if you're thinking, "What if I want to reduce redundancy and save some storage space?" That's where the **Snowflake Schema** comes in. It's like the Star Schema, but with a twist—here, the dimension tables are further broken down into related sub-tables, making the structure look like a snowflake.

How It Works:

- **Fact Table:** Still at the center, holding your key data.

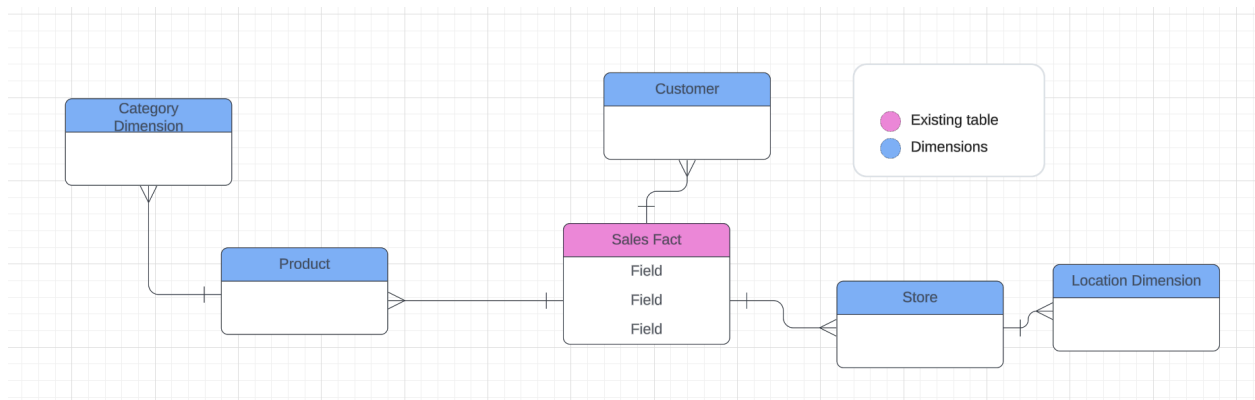
- **Dimension Tables:** These are normalized, meaning they're split into multiple related tables to reduce redundancy.

Visually, the snowflake schema looks like the star schema but with the dimension tables branching out into related tables, resembling a snowflake.



Example:

Continuing with the retail store example:



- Product Dimension might be split into Product Dimension and Category Dimension:

Product Dimension: Contains ProductID, ProductName, and CategoryID.

Category Dimension: Contains CategoryID and CategoryName.

- Store Dimension could be split into Store Dimension and Location Dimension:

Store Dimension: Contains StoreID and LocationID.

Location Dimension: Contains LocationID, City, and State.

Why Choose a Snowflake Schema?

- **Saves Space:** By normalizing your data, you cut down on duplication.
- **More Joins:** Queries can be more complex because they involve more tables, which means more joins.
- **Less Data Redundancy:** Normalized structure reduces data redundancy.

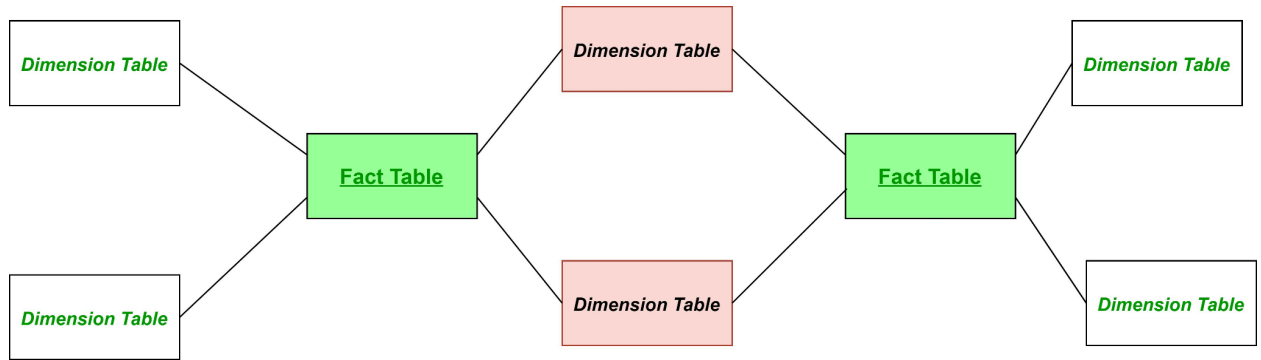
Galaxy Schema: When You Need Ultimate Flexibility

Finally, there's the **Galaxy Schema**—also known as the **Fact Constellation Schema**. Imagine having multiple fact tables that share dimension tables. This setup is like looking at a constellation of stars, where each star is a fact table connected to shared dimensions.

How it works:

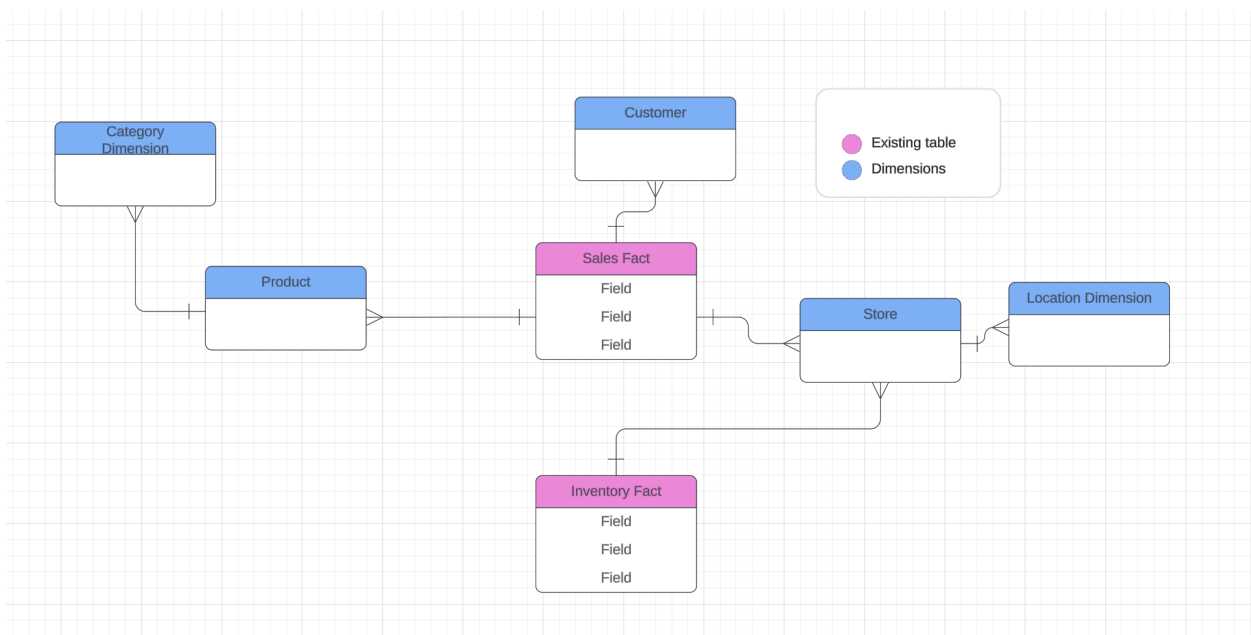
- **Multiple Fact Tables:** Represent different business processes or events (e.g., sales, shipments).
- **Shared Dimension Tables:** Provide context for multiple fact tables.

Visually, the galaxy schema looks like multiple stars (fact tables) connected to the same set of dimension tables, forming a constellation.



Example:

Imagine you are managing both sales and inventory for the retail store:



- **SalesFact:** Tracks sales data (e.g., SalesAmount, QuantitySold).
- **InventoryFact:** Tracks inventory data (e.g., StockLevel, ReorderQuantity).
- **ProductDimension:** Shared between both SalesFact and InventoryFact.
- **StoreDimension:** Also shared between the fact tables.

When to Use a Galaxy Schema:

- **Multiple Fact Tables:** Great for businesses with multiple processes or events to track (like sales and inventory).
- **Shared Dimensions:** Helps maintain consistency across different fact tables.

- **Complex Queries:** Supports advanced queries that pull in data from different processes.

Slowly Changing Dimensions (SCDs): Handling Changes Over Time

Let's switch gears and talk about how you handle changes in your dimension data. In the real world, things change—customers move, products get rebranded, and so on. To capture these changes in your data warehouse, we use what's called **Slowly Changing Dimensions (SCDs)**.

SCD allows us to model such updates to dimensions in multiple ways, each with its pros and cons. In this lecture we will delve a level deeper into data modeling and utilize these SCD techniques to handle the use case we just discussed above.

There are many approaches to deal with SCD. All these approaches are mentioned below:-

SCD Type 0: The “Do Nothing” Approach/ The Passive Method

First up is SCD Type 0, which is the most basic approach: you do nothing. That's right—no changes are captured. The original data stays exactly as it was when it was first inserted. In this method no special action is performed upon dimensional changes. Some dimension data can remain the same as it was first time inserted, others may be overwritten.

Example:

Imagine a product was initially categorized as “Electronics” and later reclassified as “Gadgets.” With SCD Type 0, the original classification “

Electronics” would remain unchanged in your dimension table.

SCD Type 1: Overwriting the Old Value

Next, we have SCD Type 1, where you overwrite the old value with the new one. Simple and straightforward, but it means you lose the historical data.

Example:

If a customer changes their last name, SCD Type 1 would overwrite the old

name with the new one. There's no history kept, so you'd only see the most recent name.

Why Use SCD Type 1?

- **Simplicity:** Easy to implement with minimal storage requirements.
- **Downside:** You lose the history of changes.

SCD Type 2: Keeping a Full History

SCD Type 2 is one of the most popular methods because it lets you keep a full history of changes. When something changes, instead of overwriting the old value, you create a new record. This way, you can track all the changes over time.

Example:

Let's say a customer moves to a new address. With SCD Type 2, you'd add a new row with the updated address, while keeping the old record intact. You'd also track when the new address became effective and when the old address was valid until.

Why Use SCD Type 2?

- **Full History:** Perfect for scenarios where historical data is important.
- **Complexity:** More complex to implement and requires more storage, but worth it for the rich historical insights.

Surrogate Keys: Managing Complex Data

In many cases, natural keys (like **ProductID** or **CustomerID**) are not sufficient to uniquely identify records, especially when tracking changes over time using SCD Type 2. This is where surrogate keys come into play.

What is a Surrogate Key?

- **Definition:** A surrogate key is an artificial, system-generated unique identifier for a row, used to differentiate between records in a table.
- **Purpose:** Surrogate keys act as primary keys and help capture all states of the data.

Why Use a Surrogate Key?

- **Non-Null and Unique:** The primary attribute of a surrogate key is that it must be non-null and unique.

- **Tracking History:** Surrogate keys are particularly useful in SCD Type 2 scenarios where multiple records with the same natural key might exist, each representing a different version of the data.

SCD Type 3: Adding a New Column

SCD Type 3 is a bit of a hybrid approach. Instead of adding a new row for every change, you just add a new column to capture the previous value. This method is useful when you only need to track the most recent change.

Example:

Imagine a product gets rebranded. With SCD Type 3, you'd add a column for the old brand name while updating the existing column with the new brand name.

Why Use SCD Type 3?

- **Simple History:** Tracks only the most recent change, making it less complex than SCD Type 2.
- **Limitations:** Not suitable for tracking multiple changes over time.

SCD Type 4: Using a Historical Table

SCD Type 4 takes a different approach by using two tables: one for the current data and another for the historical data. The main dimension table only holds the current version, while a separate history table keeps all the old versions.

Example:

Let's say a customer changes their address multiple times. The main dimension table would have only the current address, while a historical table would store all previous addresses along with the dates they were valid.

Why Use SCD Type 4?

- **Clear Separation:** Keeps your current data clean and easy to query, while the history is stored separately.
- **Complexity:** Requires maintaining two tables, which can add complexity.

SCD Type 6: The Hybrid Approach

Finally, there's SCD Type 6, which combines elements of SCD Type 1, 2, and 3. This method allows you to capture every change, keep a full history, and also track the most recent change all in one table. Ralph Kimball calls this method "Unpredictable Changes with Single-Version Overlay" in The Data Warehouse Toolkit (the bible for data modeling)...

Example:

When a product is rebranded, SCD Type 6 would create a new record (like in SCD Type 2), overwrite the current value (like in SCD Type 1), and store the previous value in a separate column (like in SCD Type 3).

Why Use SCD Type 6?

- **Comprehensive Tracking:** Ideal when you need to capture all changes, track the most recent change, and keep a history.
- **Complexity:** It's the most complex method but offers the most detailed tracking.

Designing an OLAP Data Model for Uber Rides

Let's now take everything we've learned and apply it to designing an OLAP data model for a real-world scenario: Uber rides. When designing an OLAP (Online Analytical Processing) data model for Uber rides, the goal is to organize the data in a way that supports deep analysis of ride patterns, driver performance, customer behavior, and more. Let's walk through the key components and explain how to structure this data model.

Understanding the Key Entities:

In the context of Uber rides, some of the key entities you'll need to model include:

- **Rides:** Each individual trip taken by a passenger.
- **Drivers:** The people who drive the passengers.
- **Customers (Passengers):** The people who book and take rides.
- **Time:** The temporal aspect of when rides occur.
- **Locations:** The pickup and drop-off points of rides.
- **Vehicles:** The cars used for rides.

Identifying the Key Facts:

In an OLAP model, facts represent the quantitative data you want to analyze. For Uber rides, the main fact table might include:

- **Ride Details:** This table will capture the core data about each ride.

Attributes in the Ride Fact Table:

- **RideID (Primary Key)**
- **DriverID (Foreign Key to the Driver dimension)**
- **CustomerID (Foreign Key to the Customer dimension)**
- **VehicleID (Foreign Key to the Vehicle dimension)**
- **PickupLocationID (Foreign Key to the Location dimension)**
- **DropoffLocationID (Foreign Key to the Location dimension)**
- **TimeID (Foreign Key to the Time dimension)**
- **Distance:** The distance covered during the ride.
- **Fare:** The total cost of the ride.
- **Duration:** The time taken to complete the ride.

Identifying the Key Dimensions:

Dimensions provide the context for the facts. In the case of Uber rides, the key dimensions might include:

1. **Time Dimension:**
 - **Attributes:** TimeID (Primary Key), Date, Day, Month, Quarter, Year, Hour, Minute.
 - **Purpose:** Allows analysis of rides over different time periods, such as peak hours, weekends, or specific months.
2. **Driver Dimension:**
 - **Attributes:** DriverID (Primary Key), Name, Age, Gender, License Number, Rating, Experience (in years).
 - **Purpose:** Helps analyze ride data based on driver characteristics, such as performance by experience level.
3. **Customer Dimension:**
 - **Attributes:** CustomerID (Primary Key), Name, Age, Gender, Account Creation Date, Total Rides Taken.
 - **Purpose:** Enables segmentation of rides by customer demographics and behavior.
4. **Location Dimension:**
 - **Attributes:** LocationID (Primary Key), Address, City, State, Zip Code, Latitude, Longitude.

- **Purpose:** Provides geographical context to rides, allowing analysis of popular pickup and drop-off points.
- 5. **Vehicle Dimension:**
 - **Attributes:** VehicleID (Primary Key), VehicleType, Make, Model, Year, License Plate Number, Capacity.
 - **Purpose:** Allows analysis based on the type of vehicle used, such as rides by car type or model.

Designing the Schema:

For OLAP systems, the schema design typically revolves around a Star Schema or a Snowflake Schema. Here's how you might structure the Uber rides data model.

Star Schema Design:

- **Central Fact Table:**
 - **Ride Fact Table:** Contains foreign keys to the Time, Driver, Customer, Location, and Vehicle dimensions.
 - Stores measures like Distance, Fare, and Duration.
- **Surrounding Dimension Tables:**
 - **Time Dimension Table:** Linked by TimeID.
 - **Driver Dimension Table:** Linked by DriverID.
 - **Customer Dimension Table:** Linked by CustomerID.
 - **Location Dimension Table:** Linked by PickupLocationID and DropoffLocationID.
 - **Vehicle Dimension Table:** Linked by VehicleID.

Schema Example:

Time Dimension

TimeID
Date
Hour
Month

|

Driver Customer Location Vehicle
Dimension Dimension Dimension Dimension

DriverID	CustID	LocID	VehicleID
Name	Name	Address	Make
Rating	Gender	City	Model

|

Ride Fact Table

RideID
DriverID
CustID
VehicleID
PickupLocID
DropoffLocID
TimeID
Distance
Fare
Duration

Use Case Scenarios:

With this schema in place, you can perform various analyses:

1. **Ride Trend Analysis:** Analyze the number of rides over time, identifying peak hours, days, or months.
2. **Driver Performance:** Compare driver performance metrics such as average fare, distance, and customer ratings.
3. **Customer Behavior:** Segment customers based on ride frequency, preferred times, or popular routes.
4. **Location-Based Insights:** Determine the most popular pickup and drop-off locations, useful for optimizing driver placement.

Advanced Schema: Snowflake and Galaxy

Depending on the complexity of your data and your analysis needs, you might consider using a Snowflake Schema or a Galaxy Schema.

- **Snowflake Schema:** The snowflake schema could be used to normalize dimensions further. For example, the **Location** dimension might be split into multiple related tables for **City**, **State**, and **Country**.
- **Galaxy Schema:** This schema could be employed if multiple fact tables are required. For instance, you might have a separate fact table for **Cancelled Rides** that shares the same dimensions as the **Ride Fact Table**.